

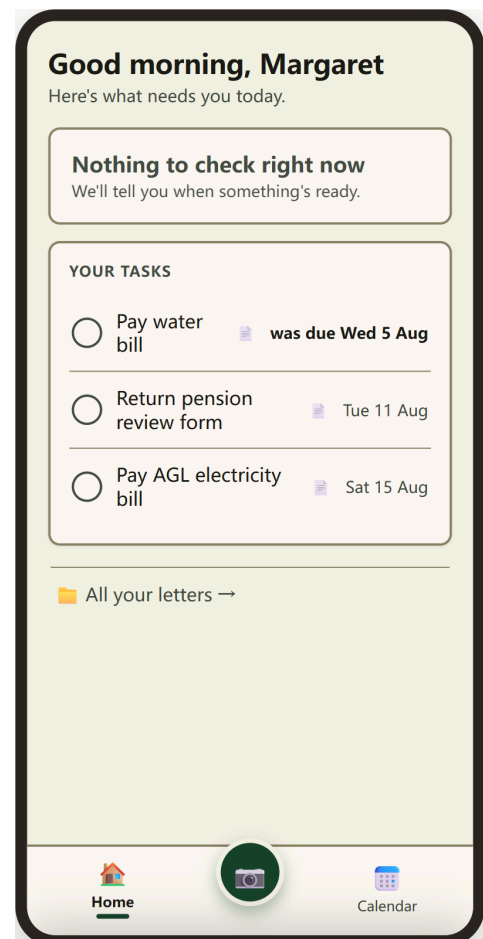
DayKeeper

A walk through the product, and the machinery behind each screen

This document follows one letter from the moment it is photographed to the moment the task it became is ticked off. Every screen you see is the working prototype; under each one is what the system does, which tables it writes, and why it was designed that way rather than another way.

It describes the shape of the system, not the state of the work. The board says who is doing what this week; this says what the thing is.

Written by Junchun Zhang, technical lead
For Dr Golnoush Abaei (supervisor), and the team
Screens the clickable prototype, docs/prototype/user/
Version 1.0



The letter by the front door

Margaret Wilson is 78. She lives alone in a brick house in the northern suburbs of Melbourne, her cataracts are getting worse, and every bill she has still arrives on paper. Her daughter is in Sydney and helps by phone. She has a smartphone, and she uses it for calls and for photographs.

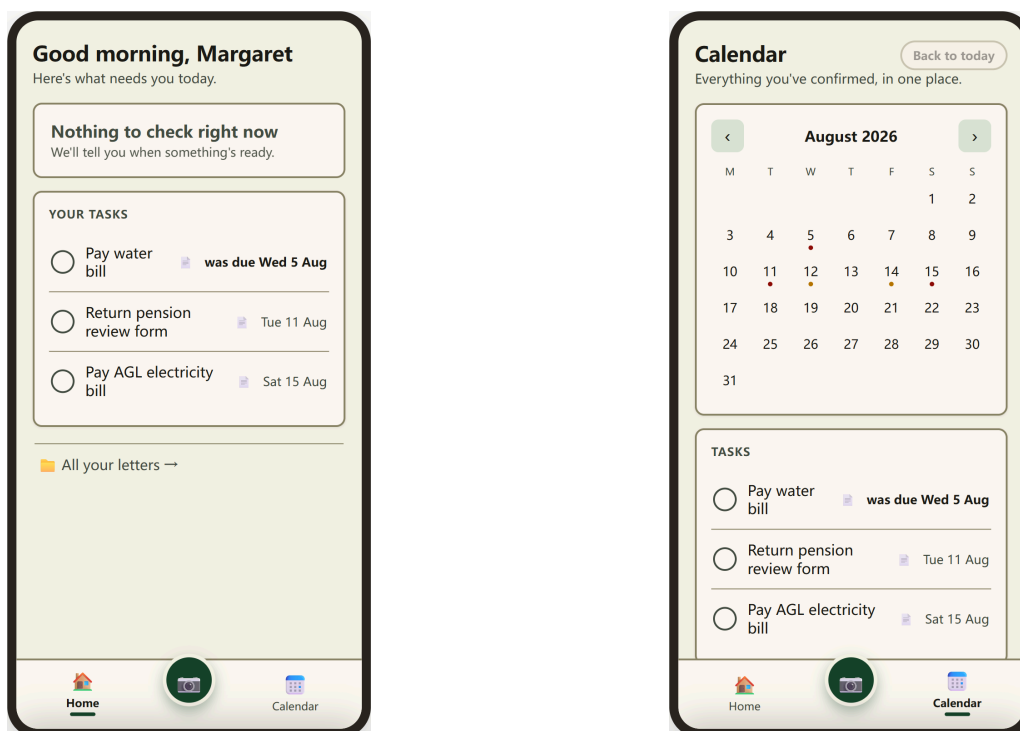
A letter arrives. It is a bill, or a form, or an appointment. Somewhere in it is a date, and somewhere else is the one thing she has to do before that date. The letter is designed to be read by a person with time and good light, and it will sit by the front door until she has both.

What goes wrong is not that she cannot read it. It is that reading it is not enough. The date has to survive the week between reading the letter and acting on it, and nothing in the house is doing that job. Her daughter's phone calls are doing that job.

This product does one thing: it makes the date outlive the letter. She photographs the letter, and the date it carries becomes a task, a place on a calendar, and three reminders. That is the whole of it.

One thing to know before the screens. The prototype has a fixed fictional today: **Monday 10 August 2026**. Every date in every screenshot is relative to it, which is why the water bill due on the fifth is already late and the pension form due on the eleventh is not. The prototype wears this date in its toolbar so nobody has to guess.

What she gets back



The list, and the calendar drawn from it. Overdue is told in words at the top rather than in red, because colour is never the only signal here.

One list of what to do and the day it has to be done, and a calendar drawn from that same list. From where she is standing, that is the whole product.

The way she operates it is **two verbs: photograph, and tick**. She never types a date, never sorts pages, never files anything, never names a document, and never opens a settings screen to make any of it work. Everything else in this document exists to keep that true.

Nothing happens until you say so, and never from a date you haven't checked.

That promise is on the review screen, and it is the sentence this document keeps coming back to. The interesting part is that it is not kept by being careful. It is kept by there being no path.

Who she is matters to the design more than usual, because the people this product is for are the ones least able to absorb a mistake it makes. Eyes over seventy, a memory that is not reliable, and a lifetime of assuming that when a computer says no it is their own fault. So body text clears 7:1 contrast rather than the usual 4.5:1, nothing is blue, colour is never the only signal, and an error message never opens by describing something she did.

What this release is, and what it is not

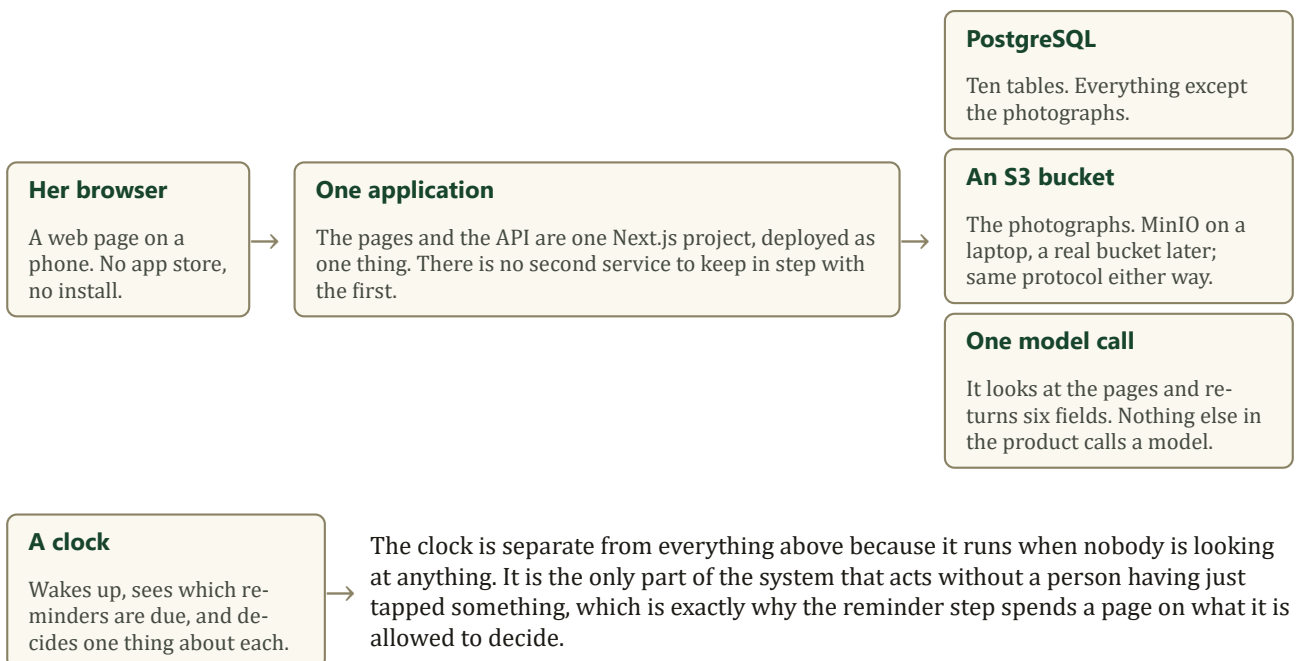
This is the first release, and it is deliberately narrow. One upload is one letter. It may run to several pages, and every photograph in it belongs to that one letter, because she said so by taking them together.

That sentence is doing a lot of work. A larger design accepted a pile of mixed post at once, worked out where one letter ended and the next began, and joined a later upload to a letter already in the system. Each of those needed its own model, its own prompt, its own way of being wrong quietly. Removing them is what turns the rest of this document into something a small team finishes.

What is left is one model call whose answer has a fixed shape, and after that a system made of tables, dates and a checkbox. The full list of what is in and what is deferred is `docs/scope.md`, which every other document in this repository defers to.

The pieces

Six things do all the work. Everything later in this document is one of them doing its job.



One application rather than two. The project description names React and Next.js in the same breath as Python and FastAPI, which reads as an invitation to build a front end talking to a Python API. The scaffold this project stands on was already a single Next.js application, and the decision was to keep it that way. A second service is a permanent tax: two dependency sets, two deployments, a network boundary to authenticate across, and a second set of types kept identical to the first by hand. The one honest argument for Python is

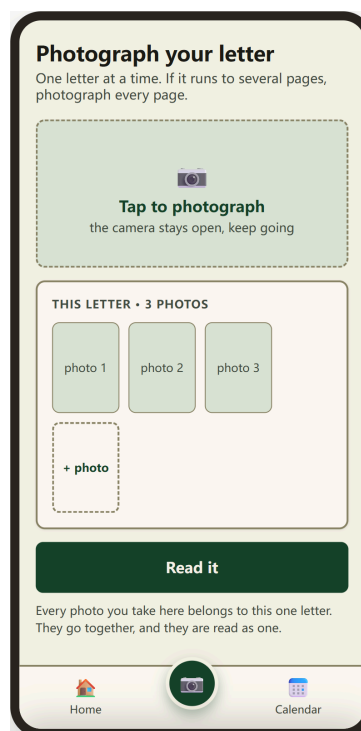
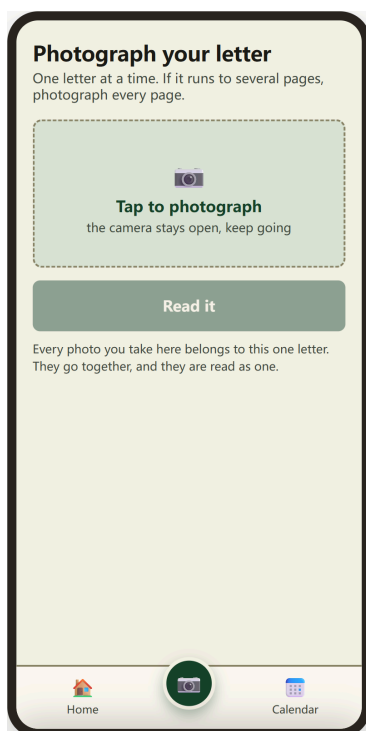
that the AI ecosystem lives there, and it does not apply here, because calling a vision model is an HTTP request in any language.

One letter, from the front door to the tick

Six steps. Each one is a screen she sees and a row somewhere she does not, and the point of this part is to put those two things next to each other. Where a step writes nothing, that is said too, because a step that writes nothing is usually a step that was designed not to ask her anything.

STEP 1 She photographs the letter

Writes: documents document_pages



The camera stays open across taps, so a three page letter is three taps rather than three trips. The button stays dim until there is at least one page.

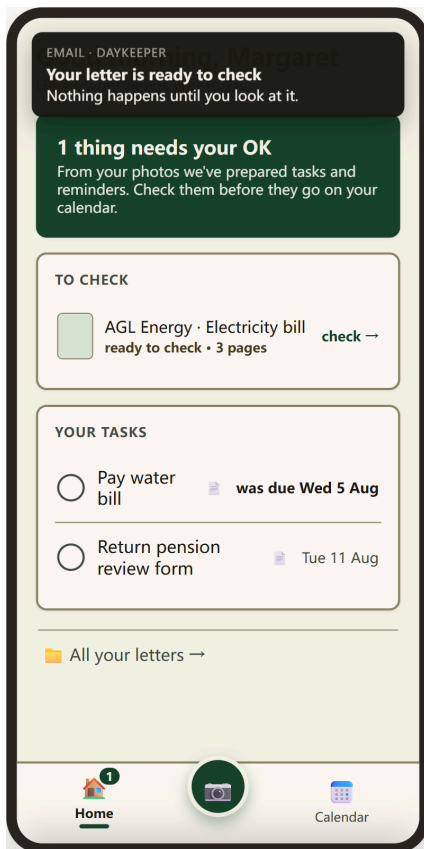
The screen says one letter at a time, and it means it. Every photograph taken here belongs to the same letter, and the system takes that as given rather than working it out later.

The cheapest way to know which letter a page belongs to is to ask nobody. She already knows, because she is holding the letter. Taking the pages together is her saying so, and it costs her nothing she was not already doing. The alternative is a model looking at nine photographs and guessing where one letter ends, which is a real problem with a real error rate, and this release does not have it.

The photographs go to an object store rather than into the database, and the row in document_pages holds the key to find them again. A photograph is large, it never changes, and it is served straight from the bucket to the browser rather than through the application.

STEP 2 The system reads it

Writes: extraction_runs extracted_fields ai_prompt_logs



This is the only place in the product where a model is called. It takes about ten seconds, which is long enough that holding her on a loading screen would be a small cruelty and short enough that a queue would be over-engineering. So the request answers immediately, the interface polls, and the home screen carries the waiting.

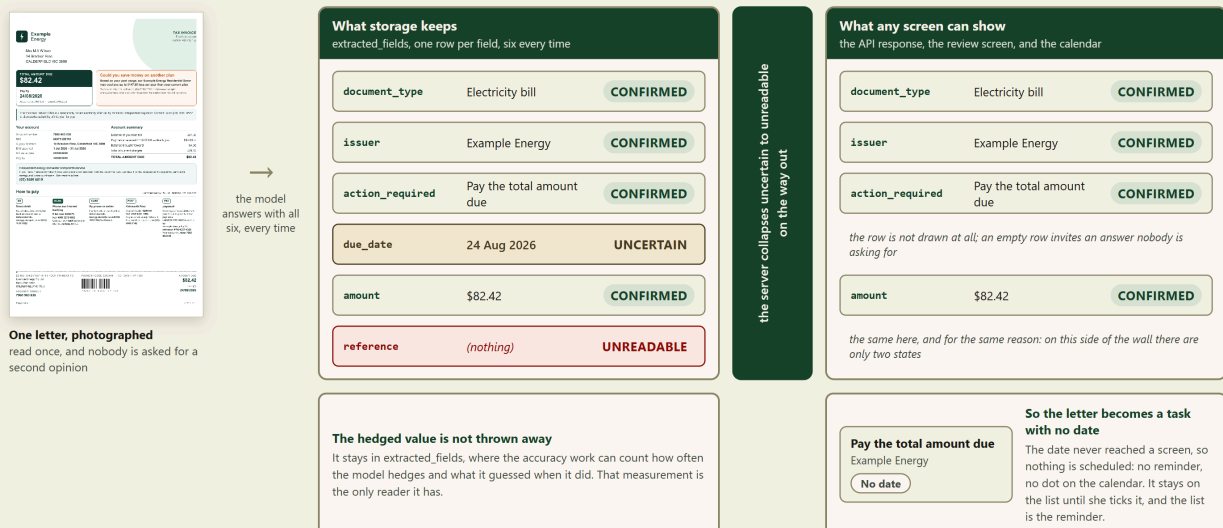
When it finishes, a notification arrives. The wording is deliberate: it says the letter is ready to check, and that nothing happens until she looks at it. It is an invitation, not a report of something already done.

What comes back is six fields, each with a status. The definition of those six, and the rule that a reader may return more but never fewer, is in `src/lib/contract/fields.ts`.

The notification, and behind it the letter waiting to be checked. The waiting was on the home screen, not on a spinner, so she could close the app.

Six fields, three states, and the one she never sees

One letter is read once and comes back as six fields, each carrying a status. What the database keeps and what any screen is allowed to show are not the same list, and the difference is made in one place, on the server, on the way out.



Trusting the model includes trusting its "I am not sure". When it hedges, the product takes the hedge at face value instead of asking her to overrule it. Storage keeps the hedged value, because how often the model hedges and what it guesses is evaluation data. Nothing else keeps it: no API response, no screen, no calendar.

As far as the product behaves, a value the model was unsure of does not exist. The calendar therefore has exactly two sources, a confident reading that a person has seen, or nothing, and no procedure has to guard against a third.

Three statuses live in the database and two reach a screen. The collapse happens once, on the server, on the way out.

A value the model was not sure of is not offered as a guess. It arrives at the screen as no value at all, and the card says so in a sentence. The reason is the promise on the review screen: a date she has not checked has no path to the calendar, and a half sure date shown in grey is a date she will check by nodding.

STEP 3 She checks it

Writes: *nothing at all*

Check what we found
AGL Energy · Electricity bill

WHAT IT SAYS

From	AGL Energy
What to do	Pay the amount due
Due date	15 Aug 2026
Amount	\$347.60
Reference	9201 4471 88

WHAT DAYKEEPER WILL DO

- 🔔 Remind you: Wed 12 Aug, 9 am
- 🔔 Remind you: Fri 14 Aug, 9 am
- 📅 On your calendar: due Sat 15 Aug

Nothing happens until you say so, and never from a date you haven't checked.

Looks right, save it

Not now

Home Calendar

This is the screen the product is built around, and it is the one that writes nothing at all.

There are no inputs here on purpose. Her job is recognition, not data entry: does this match the letter in her hand? Recognition is something a tired person with poor eyesight can still do reliably. Entry is not.

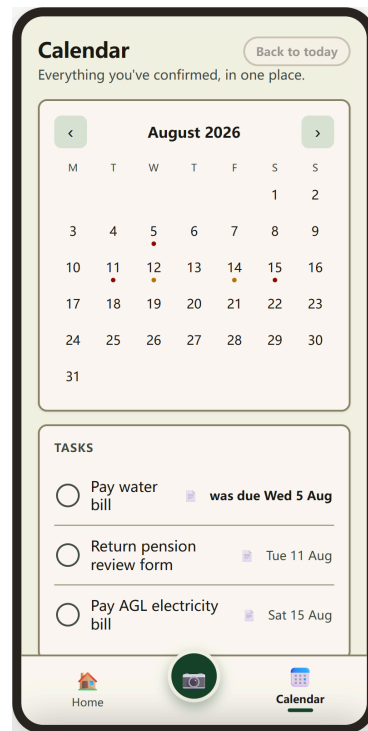
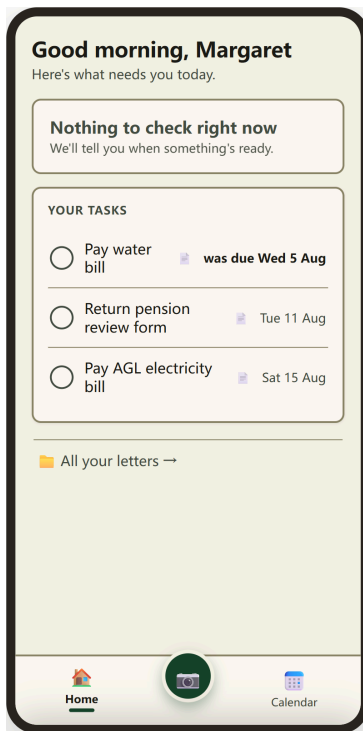
Under **what DayKeeper will do** is the plan, before it is a plan. Two reminder mornings and a calendar entry, named as dates rather than as settings. She is agreeing to a specific week, not to a feature.

There are two reminders here rather than three because the bill was confirmed on the tenth and is due on the fifteenth. The seven day rung fell in the past, so it is not offered. The card and the rows written afterwards are planned by the same function so that the screen can never promise a reminder that never arrives.

The whole screen. Nothing on it is editable and nothing on it is a question.

STEP 4 Her yes writes the rows

Writes: **tasks** **reminders** **documents**



The same letter, now a task on the list and three marks on the calendar: two reminder mornings in gold and the due date in red. Confirming is the only thing in the product that turns a reading into something that will act on its own. Before it, the letter is a row and some fields. After it, there is a task with a date and three alarm clocks set.

The calendar has exactly two sources: a confident reading a person has seen, or nothing. There is no third path, no import, no inference from an unconfirmed letter. That is why the promise on the previous screen holds without anyone having to be careful.

STEP 5 A reminder rings

Writes: reminders



Any day on the calendar opens like this. A reminder is a fact about a morning, not a notification setting.

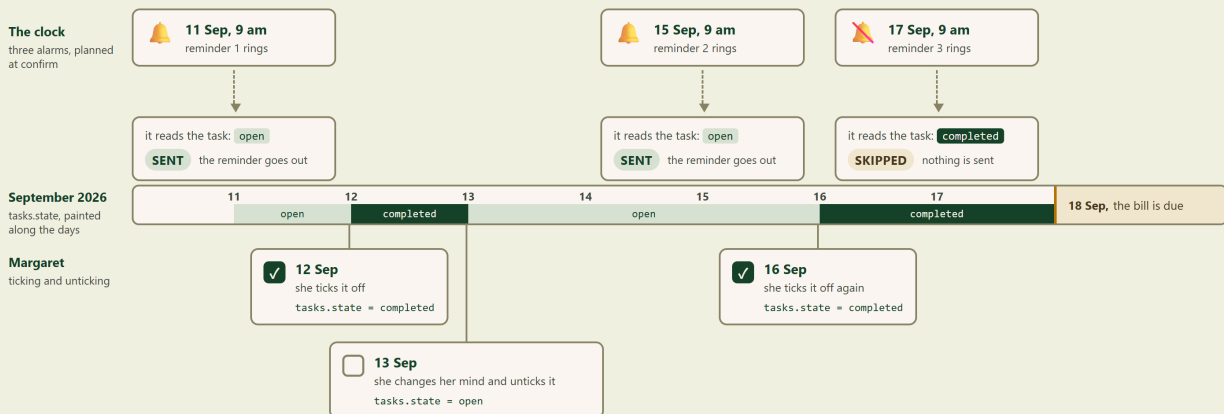
Seven days, three days and one day before, at nine in the morning. Every letter gets the same three, and a rung that would fall in the past is simply not planned.

The rows are written at confirm time rather than worked out later, because each one has a fate of its own to record: sent, skipped, or failed. The calendar draws its gold marks from those rows.

When one rings, it reads the task at that moment and decides right then whether to speak. Still open means send it. Already ticked means send nothing and write that down.

The clock checks the tick at the moment it rings

One bill, due 18 September, confirmed on 11 September. Ticking it off changes one column of one row and writes nothing to the reminders table. Each reminder is an alarm clock that looks at the task at the moment it rings and decides right then whether to speak. So Margaret can change her mind as often as she likes, and nothing has to be kept in step.



All three of her actions write to the same column of the same row, **tasks.state**. None of them reads the reminders table, and none of them writes to it.

reminders		written once, at confirm
scheduled_for	status	who wrote the status
2026-09-11 09:00	SENT	the dispatcher, at 9 am on the 11th
2026-09-15 09:00	SENT	the dispatcher, at 9 am on the 15th
2026-09-17 09:00	SKIPPED	the dispatcher, at 9 am on the 17th

Three rows, planned when she confirmed the letter. Ticking wrote none of them; the dispatcher wrote the last column, each row at its own moment.

The design this replaced

REMOVED

Ticking cancelled every waiting reminder, unticking revived them, except the ones whose time had already passed, which had to stay cancelled. Two writes, then two writes back, minus an exception. All of it bookkeeping, and the undo rule needed an exception within a week of being written.

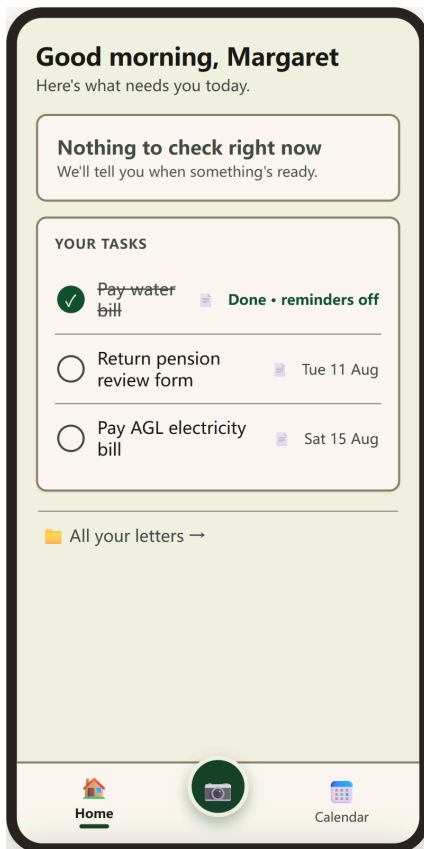
A task stores one fact a person controls: whether it is ticked. Overdue, upcoming and completed are worked out while drawing the screen, from that tick and today's date in her timezone. Nothing writes the word "overdue" anywhere, so nothing has to wake up at midnight to write it, and nothing can forget to unwrite it. **Ticking is never locked:** tick at breakfast, untick at lunch, tick again at dinner, three writes to one column, no ledger and no residue.

Ticking writes one column of one row and touches nothing in the reminders table. The judgement is made at the moment of use, not stored in advance.

The alternative was bookkeeping. Ticking would cancel every waiting reminder, unticking would revive them, except the ones whose time had passed. That is the same truth written in two tables, and every copy needs a transaction to keep it honest and an undo rule to unwind it. Reading the tick when the alarm rings stores the truth once, so she can tick at breakfast, untick at lunch and tick again at dinner without the system keeping a ledger of her wavering.

STEP 6 She ticks it

Writes: **tasks**



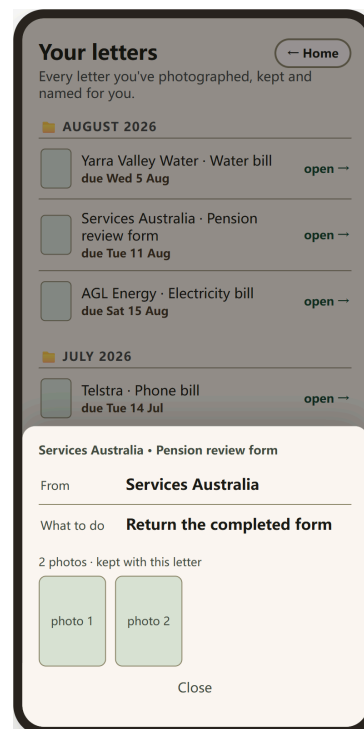
The tick is a task's only state. Whether it is overdue is not stored anywhere; it is worked out from the clock at the moment somebody asks, in Melbourne time, so a task does not become overdue at eleven at night because a server somewhere is in another timezone.

A ticked task stays on the list for a week rather than vanishing, because vanishing is indistinguishable from having been lost. Its calendar entry stays forever, so the answer to "did I pay that" is still there in December.

"Reminders off" is derived, not a switch. It is true because the tick is set, and it becomes false again the moment she unticks, with nothing to undo.

Struck through, still on the list, and told in words: done, reminders off.

The letters she has kept



Every letter that has been read, and one of them opened with the photographs that came with it.

Nothing in this area asks anything of her. It exists because a letter is evidence, and because the photographs are the only copy of a piece of paper that may well have gone in the bin. Opening a letter shows what was read and the pages it was read from, which is also the only way to check the system against the original.

When the reading fails

About one letter in eight cannot be read. The mock reader fails deliberately at roughly that rate so the path is never untested.

In this release a failed reading is the end of the road. There is no retry, no prompt to photograph it again, and no message suggesting the photograph was her fault. The letter keeps its photographs and says plainly that it could not be read.

That is a smaller promise than the product would like to make, and it is made honestly. An earlier design offered her a way out of every failure: photograph it again, or a screen explaining which kind of failure it was. Every one of those paths needed the system to know why it had failed, and a model that cannot read a page is usually not able to say why. Offering a remedy that does not work is worse than offering none, so this release offers none and says so.

Where the model is called

One place in the whole product calls a model. Everything downstream of it is ordinary code and ordinary SQL, which is worth saying plainly because it is the opposite of how a product like this is usually imagined.

Where the model is called, and where it is not

One place in the whole product calls a model. Everything after it is ordinary code and ordinary SQL, which is why the interesting engineering is concentrated in a single request and the rest of the system can be read without thinking about a model at all.

The reading

Runs once per upload, on the pages of one letter.

IN

every photograph in the upload, as images

All of them are pages of one letter. Which letter a page belongs to was settled by the person who photographed it, not inferred here.

WHAT IT DOES

Looks at the pages and answers in a fixed shape

document_type · issuer · action_required

what the letter is, who sent it, and the one thing it asks of her.

due_date · amount · reference

the three that turn a letter into something a calendar and a bank can act on, plus an optional due_time when the page prints one.

OUT

six fields, each with a status

Confirmed or unreadable. A value the model was not sure of is not offered as a guess, and the screen says so in a sentence.

vision tokens, the expensive part of the product

the photographs are looked at once and once only

about ten seconds

Why a vision model rather than reading the text first. A letter is designed for a human eye. Its meaning is carried by position, by a box drawn around the number that matters, by a heading in bold, and by five payment methods printed side by side, each with its own reference number and nothing but a column to say which belongs to which. Text extraction keeps the characters and throws away the arrangement, and the arrangement is where half the meaning was. Reading the words first and paying a second model to put the meaning back is discarding the evidence and then buying a guess.

Not a model call

these look like judgement, and every one of them is ordinary code

Whether a task is overdue

→ a date comparison in Melbourne time, made when it is asked for

Whose letters get read back

→ a WHERE user_id = ... clause the model cannot set

When a reminder goes out

→ seven, three and one day before, arithmetic on the due date

Whether to send that reminder

→ read the tick at the moment it rings, and send nothing if it is set

Whether the upload can be read at all

→ the file is an image, and it is not larger than the limit

Which letter a page belongs to

→ the upload it arrived in, decided by the person before it was sent

The single call, and six things that look like judgement and are not.

Why a vision model rather than reading the text first. A letter is designed for a human eye. Its meaning is carried by position: a box drawn around the number that matters, a heading in bold, and five payment methods

printed side by side, each with its own reference number and nothing but a column to say which belongs to which. Text extraction keeps the characters and throws away the arrangement, and the arrangement is where half the meaning was. Reading the words first and then paying a second model to put the meaning back is discarding the evidence and then buying a guess.

The bottom half of that diagram is the more useful one. Six questions that sound like they need judgement, and every one of them is a line of ordinary code: whether a task is overdue, whose letters get read back, when a reminder goes out, whether to send it, whether the upload can be read at all, and which letter a page belongs to. The last of those used to need a model. It does not any more, because she settles it by taking the photographs together.

The ten tables

The next page is the whole database at once, laid out along the same journey as the previous part. Two things in it are worth knowing before you look, because both are absences and an absence is easy to read past.

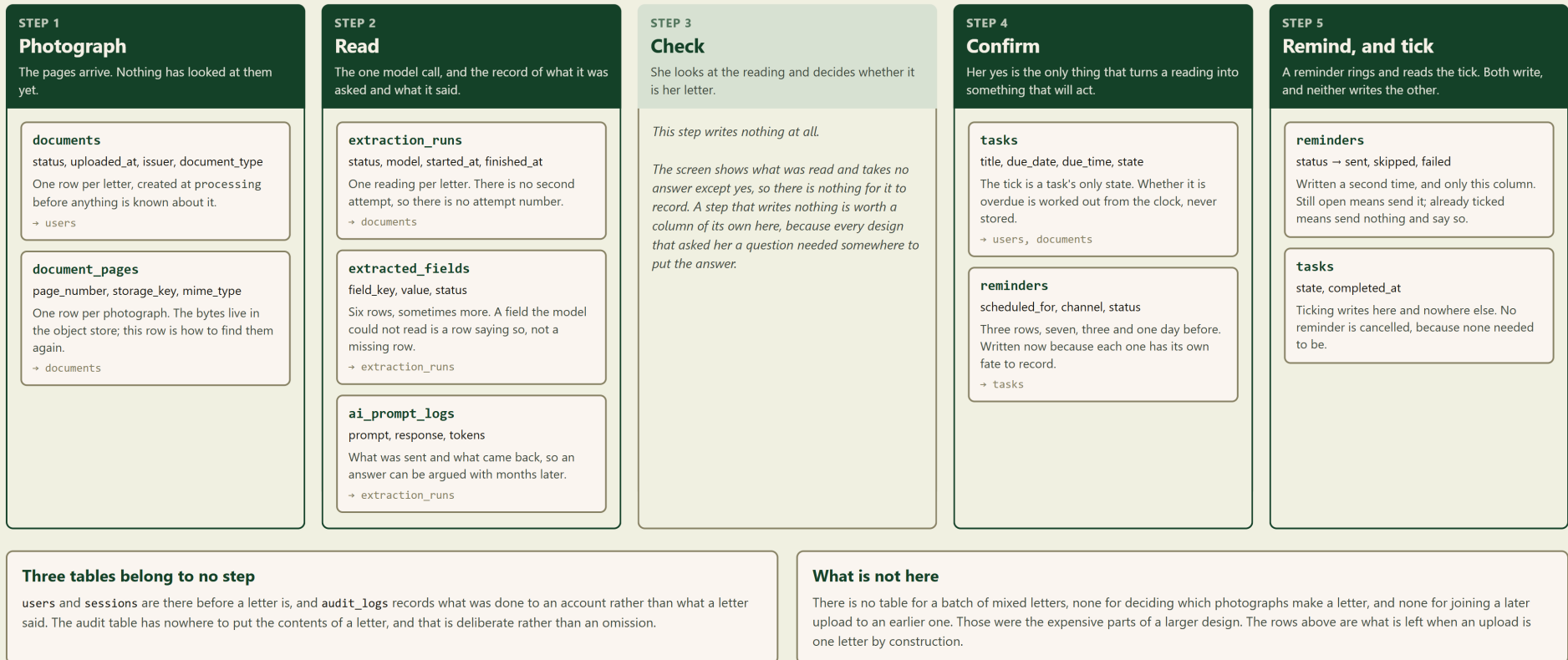
The third column is empty. The step where she checks the reading writes nothing, because the screen shows and never asks, so there is no answer for it to record. Every design that asked her a question needed somewhere to put the answer, and this one does not.

Nothing derivable is stored. There is no overdue column, no reminders-off flag, and no count of anything. Each of those would be a second copy of a truth that already exists, and a second copy is a thing that can disagree with the first. Whether a task is overdue is a comparison made at the moment somebody asks, and whether its reminders are off is simply whether the tick is set.

The definition of every column, and the reason it exists, is in `db/schema.sql`. That file is the database rather than a description of it: there are no migrations, so editing it and running `npm run db:reset` is the whole of changing the schema.

The ten tables, and which step writes each

A letter moves through the steps of the previous part, and each one writes a different part of the database. Nothing is written twice, and nothing that can be worked out at the moment it is asked for is stored. The definition of every column, with the reason it exists, is in `db/schema.sql`.



The last two steps share a column because they write at the same moment and neither writes the other: the reminder records what it decided, and the tick records that she is done.

What is not settled

This document describes a design, and a design has edges. These are the ones that are known rather than the ones that are hidden.

Open question	What is known about it
Who runs the clock	The reminder rows and the rule for reading the tick are settled. What actually wakes up at nine in the morning, and where it runs, is not.
How a reminder reaches her	The row carries a channel. In app is drawn on the screens; email needs a sender the project does not have yet, and so does password reset.
The upload at its limits	A page count and a size are checked. What the screen does when a photograph is enormous, or when the connection drops halfway, is not drawn.
Rate limiting on sign in	The password hashing is deliberate and slow. The number of attempts is not yet bounded.
Which model	The reading is written against an interface rather than a provider, so the choice is a swap of one file. Which model, and how it is paid for, is being settled outside this document.

Where the answers live

Every fact in this repository has one home, and the home is usually the thing itself rather than a document about it. This table is how to find it. If a document and the code disagree, the code is right, which is why the code is where the reasoning was put.

If you want to know	Open
what this release builds, and what it does not	<code>docs/scope.md</code>
how to run the whole thing	<code>docs/start-here.md</code>
what each endpoint takes and returns	<code>docs/api.md</code>
the six fields, and why six is a floor	<code>src/lib/contract/fields.ts</code>
what a field status means, and <code>open_payload</code>	<code>src/lib/contract/extraction.ts</code>
why the review screen has no inputs	<code>src/lib/contract/api.ts</code>
the reminder ladder, and the clock check	<code>src/lib/contract/reminders.ts</code>
why every date is Melbourne and date only	<code>src/lib/contract/dates.ts</code>
every table and every column, with reasons	<code>db/schema.sql</code>
what a reader has to promise	<code>src/server/extraction/provider.ts</code>
how the mock reader misbehaves, and why	<code>src/server/extraction/mock-provider.ts</code>
sessions, and why the passwords are hashed that way	<code>src/server/auth/</code>
where the photographs live	<code>src/server/storage.ts</code>
the palette, the contrast, the type sizes	<code>docs/theme.md</code>
the conventions, and why this is one application	<code>AGENTS.md</code>

About to change something? Read `docs/scope.md` first to see whether it is in this release at all, then find the file above. This document is a tour, and a tour is not a source: it is rebuilt from those files rather than the other way round.